SDA WEEK 15

Design Patterns

Syed Ahmed Khan
syed.ahmed@nu.edu.pk

DESIGN PATTERNS

Design pattern is a general reusable solution to a commonly occurring problem in software design. A design pattern is not a finished design that can be transformed directly into code. It is a description or template for how to solve a problem that can be used in many different situations. Object-oriented design patterns typically show relationships and interactions between classes or objects, without specifying the final application classes or objects that are involved.

DESIGN PATTERNS

- **Definition:** A design pattern is a proven reusable design solution to a commonly occurring problem in software design.

- **Nature:** They are not finished designs but rather templates/descriptions for solving problems

- **Purpose:** Help designers reach the right design faster by leveraging proven solutions

KEY CHARACTERATICS

- Standard solutions to common programming problems
- Techniques for making code more flexible
- High-level programming idioms
- Describe connections among program components
- Represent the shape of object diagrams or models

DESIGN PATTERNS & OOP

Design patterns extend OOP by:

- Providing higher-level abstractions
- Guiding large-scale design decisions
- Making your architecture easier to change
- Helping teams communicate (shared vocabulary: “use a Factory”, “apply Observer”, etc.)

DESIGN PATTERNS IN OTHER DESCIPLINES

- Automobile engineers don’t design cars from the ground up using only the laws of physics.
- Instead, they reuse proven design templates, refining them over time based on experience and performance.
- Software development is similar.
- Building systems entirely from scratch is costly and time-consuming.
- Design patterns enable reuse of well-tested software architectures and design solutions, helping developers avoid reinventing the wheel.

PATTERNS

Professor Christopher Alexander has famously defined patterns as:

- A pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.

REAL WORLD ANALOGY

Problem: Highway Crossing

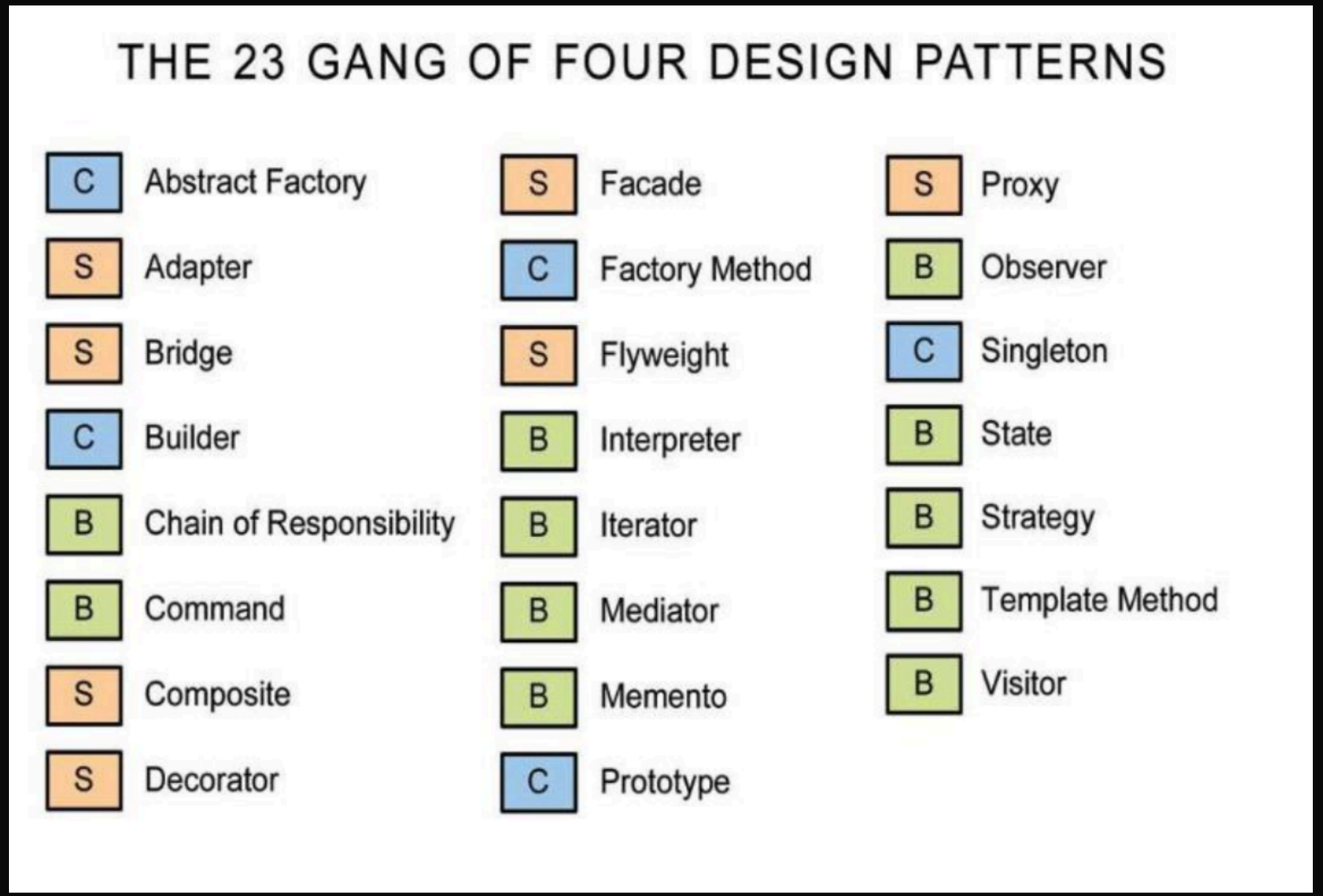
Pattern: Cover Leaf

THE GANG OF FOUR (GOF)

In 1995, four authors (Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides) published a legendary book listing 23 design patterns, the Gang of Four or GoF patterns.

- **Authors:** Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides
- **Landmark Book:** "Design Patterns: Elements of Reusable Object-Oriented Software" (1995)
- **Contribution:** Cataloged 23 fundamental design patterns in C++ and Smalltalk
- **Structure:** Book divided into OOP principles and pattern catalog

THE GANG OF FOUR (GOF)



PATTERN ELEMENTS

- **Name**
  - A meaningful pattern identifier
- **Problem description**
  - When to apply the pattern
- **Solution description**
  - Elements that make up the pattern, their relationships, responsibilities, and collaborations.
- **Consequences**
  - The results and trade-offs of applying the pattern

TYPES OF DESIGN PATTERNS

There are three basic kinds of design patterns:

- Structural
- Creational
- Behavioral

CREATIONAL DESIGN PATTERNS

Creational design patterns focus on controlling how objects are created, ensuring flexibility, consistency, and separation between creation and usage. By encapsulating the instantiation process, these patterns help systems become easier to extend, maintain, and configure.

CREATIONAL DESIGN PATTERNS

Examples of Creational Patterns:

- Factory Method
- Abstract Factory
- Singleton
- Builder
- Prototype

STRUCTURAL DESIGN PATTERNS

Structural design patterns focus on how classes and objects are composed to form larger, flexible structures. They help ensure that system architecture is efficient, scalable, and easy to refactor by defining relationships that simplify complexity.

STRUCTURAL DESIGN PATTERNS

Examples of Structural Patterns:

- Adapter
- Composite
- Proxy
- Decorator
- Facade
- Flyweight

BEHAVIORAL DESIGN PATTERNS

Behavioral design patterns define how objects interact and communicate, distributing responsibilities and enabling flexible control flows. They help make systems more modular, understandable, and adaptable to change.

BEHAVIORAL DESIGN PATTERNS

Examples of Behavioral Patterns:

- Strategy
- Observer
- State
- Command
- Iterator
- Template Method

COMMON DESIGN PATTERNS

Creational	Structural	Behavioral
• Factory • Singleton • Builder • Prototype	• Decorator • Adapter • Facade • Flyweight • Component architecture	• Strategy • Template • Observer • Command • Iterator • State

DESCRIBING DESIGN PATTERNS

- Intent
  - Short description of the pattern & its purpose
- Also Known As
  - Any aliases this pattern is known by
- Motivation
  - Motivating scenario demonstrating pattern's use
- Applicability
  - Circumstances in which pattern applies
- Structure
  - Graphical representation of the pattern using modified UML notation

DESCRIBING DESIGN PATTERNS

- Participants
  - Participating classes and/or objects & their responsibilities
- Collaborations
  - How participants cooperate to carry out their responsibilities
- Consequences
  - The results of application, benefits, liabilities
- Implementation
  - Pitfalls, hints, techniques, plus language-dependent issues

DESCRIBING DESIGN PATTERNS

- Sample Code
  - Implementations in C++, Java, C#, Smalltalk, etc.
- Known Uses
  - Examples drawn from existing systems
- Related Patterns
  - Discussion of other patterns that relate to this one

FACTORY PATTERN

TYPE: CREATIONAL DESIGN PATTERN

INTENT

- Define an interface for creating an object, but let subclasses decide which class to instantiate"
- Also known as "Virtual Constructor" pattern
- Most widely used pattern in software engineering

APPLICABILITY

- Use when:
  - A class can' t anticipate the class of objects it must create
  - Application knows about superclass but not concrete implementation at compile time
  - A class wants its subclasses to specify created objects
  - Responsibility delegation to helper subclasses needs localization

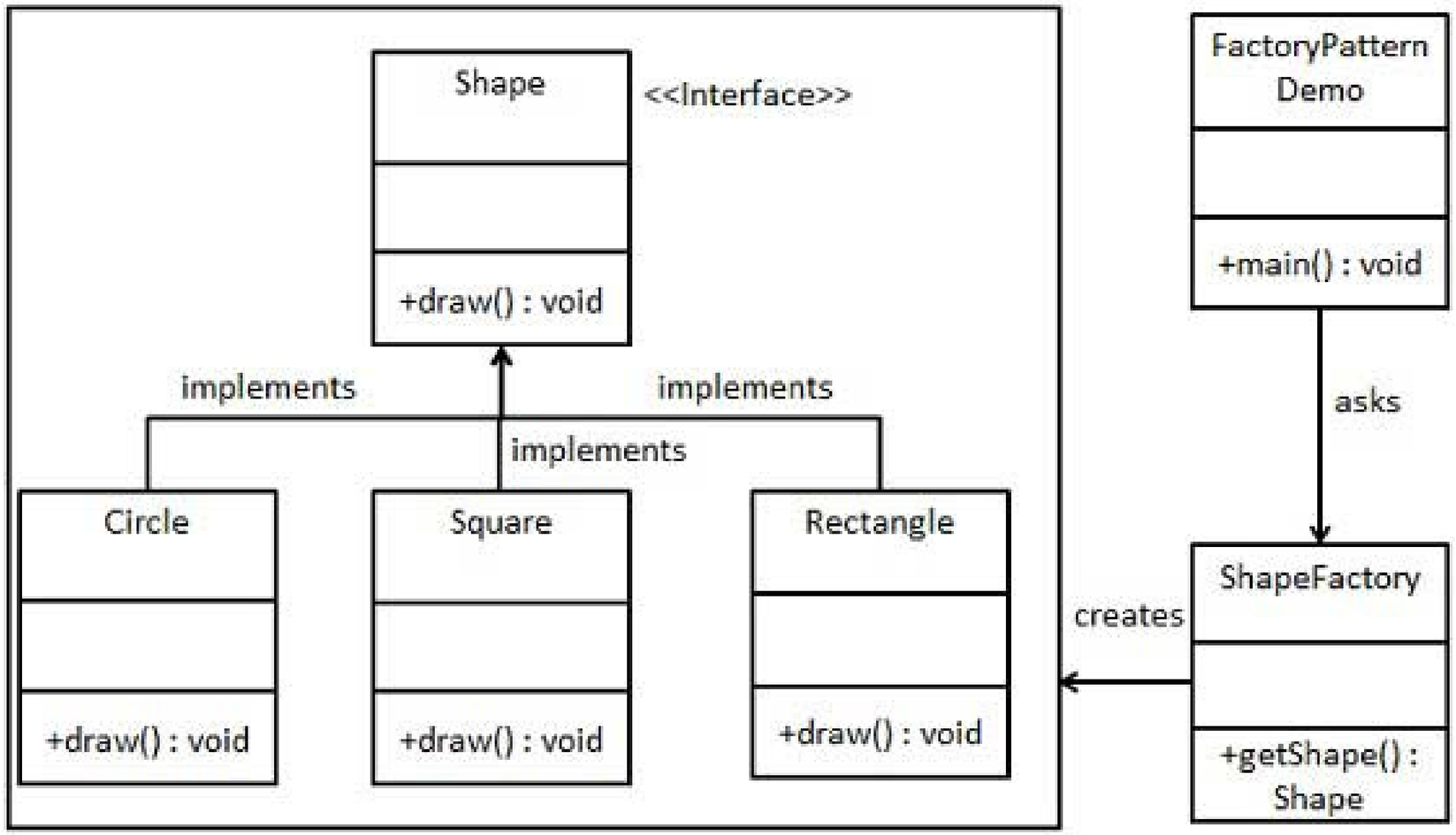
PARTICIPANTS

- **Product:** Interface of objects the factory creates
- **ConcreteProduct:** Implements product interface
- **Creator:** Declares factory method, may contain default implementation
- **ConcreteCreator:** Overrides factory method to return ConcreteProduct

EXAMPLE

We're going to create a Shape interface and concrete classes implementing the Shape interface. A factory class ShapeFactory is defined as a next step. FactoryPatternDemo, our demo class will use ShapeFactory to get a Shape object. It will pass information (CIRCLE / RECTANGLE / SQUARE) to ShapeFactory to get the type of object it needs.

STRUCTURE


Shape
<<Interface>>
+draw() : void
implements
implements
implements
Circle
+draw() : void
Square
+draw() : void
Rectangle
+draw() : void
FactoryPattern Demo
+main() : void
asks
ShapeFactory
+getShape() : Shape
creates

IMPLEMENTATION

Step 1: Create an interface.

```
//Shape.java

public interface Shape
{
void draw();
}
```

Step 1: Create an interface.

```
//Shape.cpp

class Shape {
public:
   virtual void draw() = 0;
   virtual ~Shape() {}
};
```

IMPLEMENTATION

Step 2: Create concrete classes implementing the same interface.

```
public class Rectangle implements Shape {
   @Override
   public void draw() {
      System.out.println("Inside Rectangle::draw() method.");
   }
}
public class Circle implements Shape {
   @Override
   public void draw() {
      System.out.println("Inside Circle::draw() method.");
   }
}
```

IMPLEMENTATION

Step 2: Create concrete classes implementing the same interface.

```
public class Square implements Shape {
@Override
public void draw()
System.out.println("Inside Square::draw() method.");

}}
```

IMPLEMENTATION

Step 3: Create a Factory to generate object of concrete class based on given information.

```
public class ShapeFactory {
   public Shape getShape(String shapeType) {
      if(shapeType == null) return null;
      if(shapeType.equalsIgnoreCase("CIRCLE"))
         return new Circle();
      else if(shapeType.equalsIgnoreCase("RECTANGLE"))
         return new Rectangle();
      else if(shapeType.equalsIgnoreCase("SQUARE"))
         return new Square();
      return null;
   } }
```

IMPLEMENTATION

Step 4: Use the Factory to get object of concrete class by passing an information such as type.

```
public class FactoryPatternDemo {
public static void main(String[] args)
{
ShapeFactory shapeFactory = new ShapeFactory();
Shape shape1 = shapeFactory.getShape("CIRCLE");
shape1.draw();
Shape shape2 = shapeFactory.getShape("RECTANGLE")
shape2.draw();
Shape shape3 = shapeFactory.getShape("SQUARE");
 shape3.draw();
}}
```

IMPLEMENTATION

Step 5: Verify the output

Inside Circle::draw() method.
Inside Rectangle::draw() method.
Inside Square::draw() method.

ADAPTER PATTERN

TYPE: STRUCTURAL DESIGN PATTERN

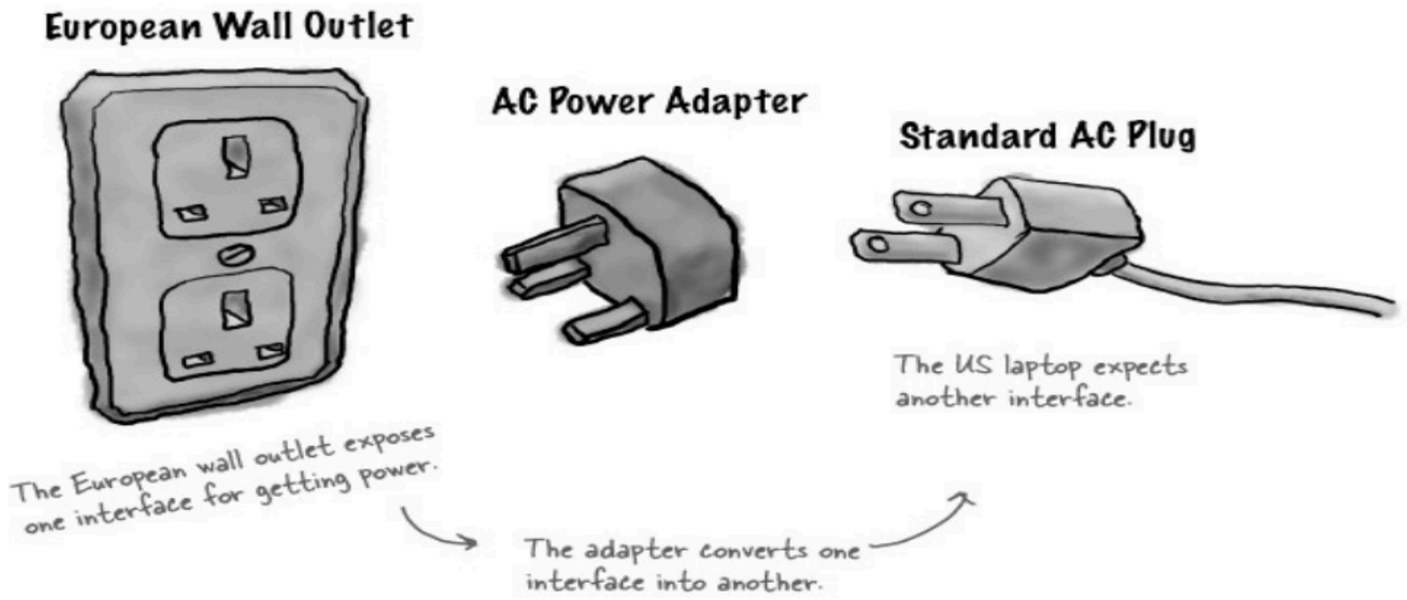
INTENT

- Convert interface of existing class into interface clients expect
- Wrap existing class with new interface
- Also known as "Wrapper" pattern

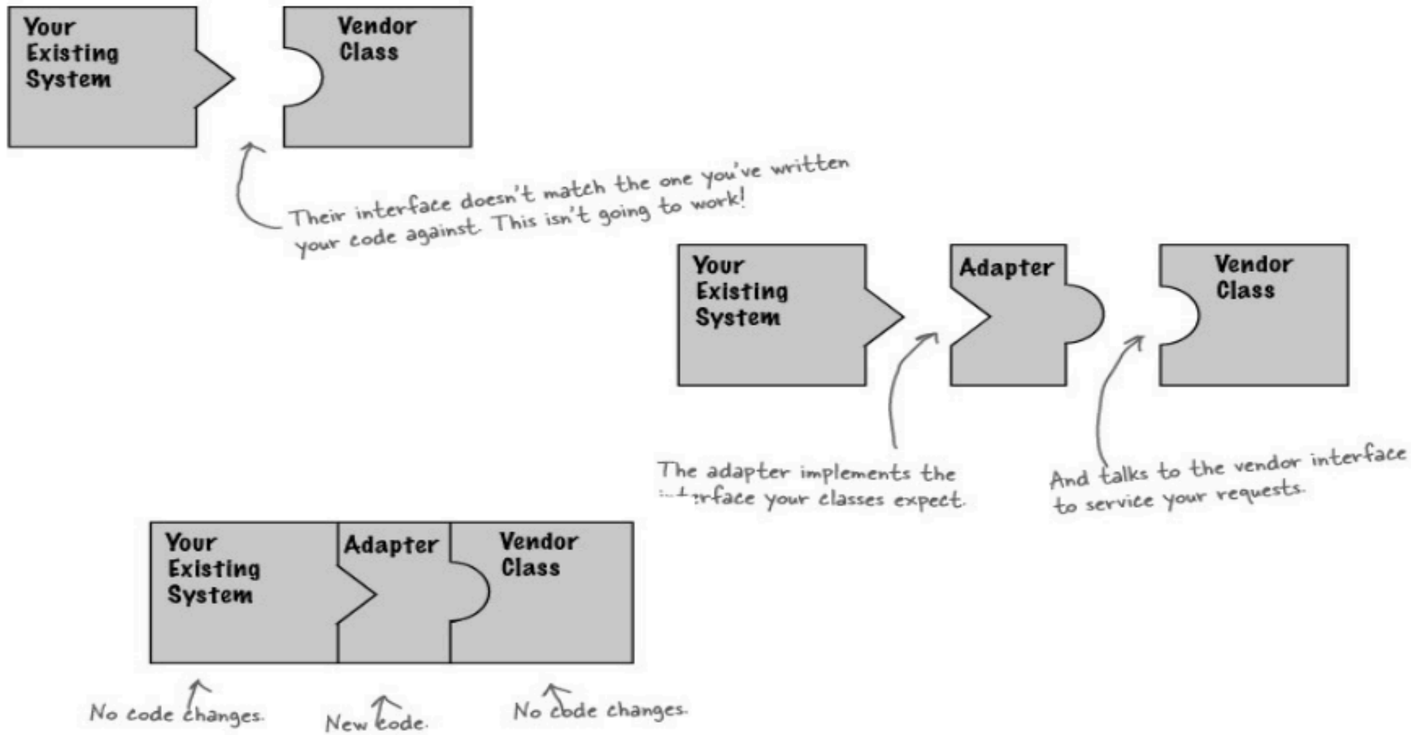
Imagine you have a power plug from Europe that has a two-pin interface, but your wall socket in Pakistan has a three-pin interface. What do you do? You use a travel adapter. The Adapter pattern does exactly this in software.

Imagine you have a power plug from Europe that has a two-pin interface, but your wall socket in Pakistan has a three-pin interface. What do you do? You use a travel adapter. The Adapter pattern does exactly this in software.

- This is an abstraction of the Adapter Pattern



ADAPTER VISUALIZED



PROBLEM & MOTIVATION

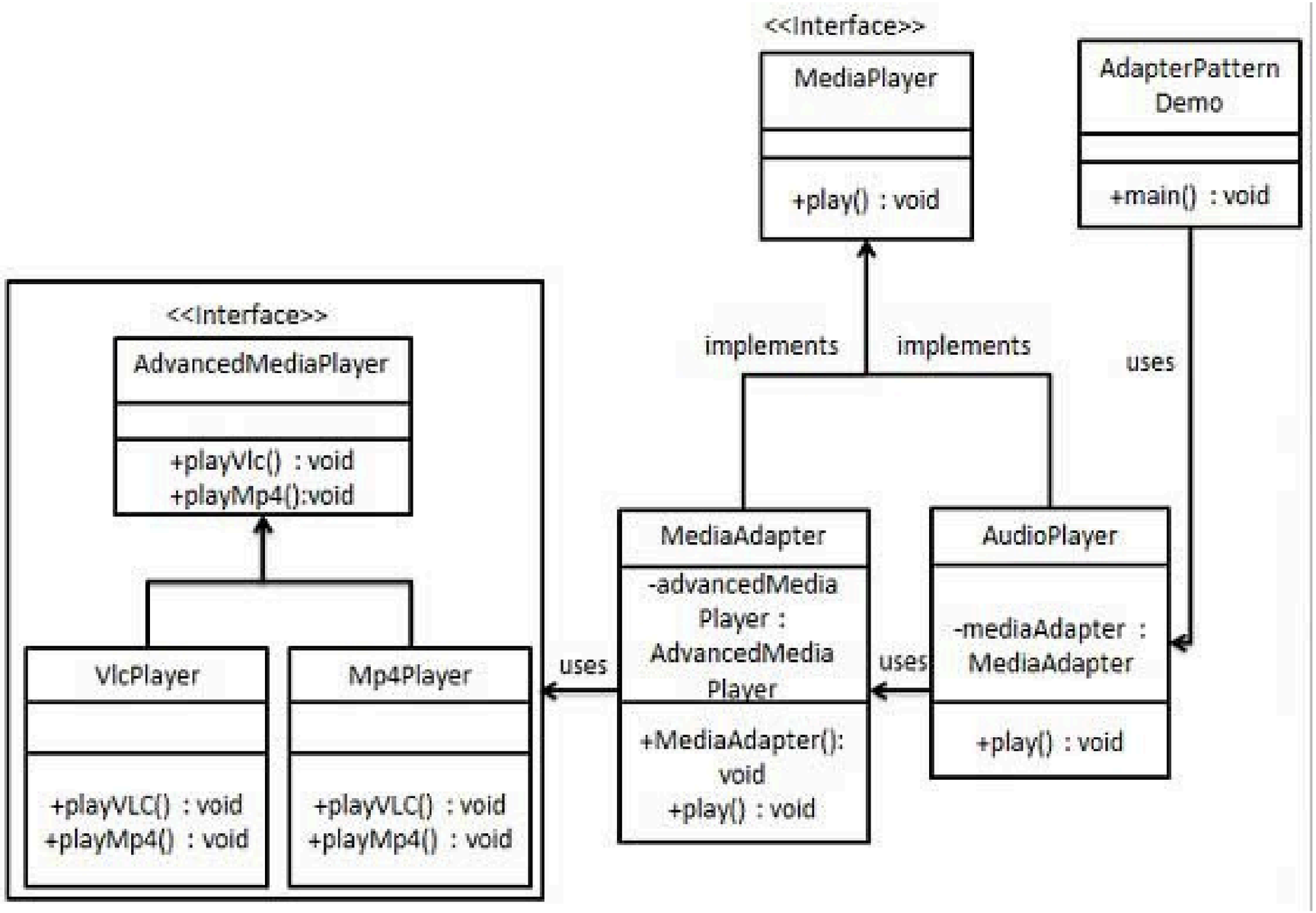
The problem is simple: you have an existing class, maybe a third-party library, a legacy system, or just a class you can't modify, that provides functionality you need. But its interface doesn't match what the rest of your code expects.

Without an adapter, you're stuck. You can't change the vendor's code, and you don't want to mess up your own clean architecture by making it conform to this weird interface. The solution? Write an Adapter class that sits between your client and the vendor class, translating the requests.

EXAMPLE

- We have a MediaPlayer interface and a concrete class AudioPlayer implementing the MediaPlayer interface. AudioPlayer can play mp3 format audio files by default.
- We are having another interface AdvancedMediaPlayer and concrete classes implementing the AdvancedMediaPlayer interface. These classes can play vlc and mp4 format files.
- We want to make AudioPlayer to play other formats as well. To attain this, we have created an adapter class MediaAdapter which implements the MediaPlayer interface and uses AdvancedMediaPlayer objects to play the required format.
- AudioPlayer uses the adapter class MediaAdapter passing it the desired audio type without knowing the actual class which can play the desired format. AdapterPatternDemo, our demo class will use AudioPlayer class to play various formats.

EXAMPLE


<<Interface>>
MediaPlayer
+play() : void
AdapterPattern Demo
+main() : void
implements
implements
uses
<<Interface>>
AdvancedMediaPlayer
+playVlc() : void
+playMp4():void
MediaAdapter
-advancedMedia Player : AdvancedMedia Player
+MediaAdapter(): void
+play() : void
AudioPlayer
-mediaAdapter : MediaAdapter
+play() : void
uses
uses
VlcPlayer
+playVLC() : void
+playMp4() : void
Mp4Player
+playVLC() : void
+playMp4() : void

IMPLEMENTATION

Step 1: Create interfaces for Media Player and Advanced Media Player.

```
public interface MediaPlayer {
   public void play(String audioType, String fileName);
}
```

```
public interface AdvancedMediaPlayer {
   public void playVlc(String fileName);
   public void playMp4(String fileName);
}
```

IMPLEMENTATION

Step 2: Create concrete classes implementing the AdvancedMediaPlayer interface.

VlcPlayer.java

```
public class VlcPlayer implements AdvancedMediaPlayer{
   @Override
   public void playVlc(String fileName) {
      System.out.println("Playing vlc file. Name: "+ fileName);
   }

   @Override
   public void playMp4(String fileName) {
      //do nothing
   }
}
```

IMPLEMENTATION

Step 2: Create concrete classes implementing the AdvancedMediaPlayer interface.

Mp4Player.java

```
public class Mp4Player implements AdvancedMediaPlayer{

   @Override
   public void playVlc(String fileName) {
      //do nothing
   }

   @Override
   public void playMp4(String fileName) {
      System.out.println("Playing mp4 file. Name: "+ fileName);
   }
}
```

IMPLEMENTATION

Step 3: Create adapter class implementing the MediaPlayer interface.

MediaAdapter.java

```
public class MediaAdapter implements MediaPlayer {
   AdvancedMediaPlayer advancedMusicPlayer;

   public MediaAdapter(String audioType){

      if(audioType.equalsIgnoreCase("vlc") ){
         advancedMusicPlayer = new VlcPlayer();

      }else if (audioType.equalsIgnoreCase("mp4")){
         advancedMusicPlayer = new Mp4Player();
      }
   }
   @Override
   public void play(String audioType, String fileName) {

      if(audioType.equalsIgnoreCase("vlc")){
         advancedMusicPlayer.playVlc(fileName);
      }
      else if(audioType.equalsIgnoreCase("mp4")){
         advancedMusicPlayer.playMp4(fileName);
      }
   }
}
```

IMPLEMENTATION

Step 4: Create concrete class implementing the MediaPlayer interface.

AudioPlayer.java

```java
public class AudioPlayer implements MediaPlayer {
   MediaAdapter mediaAdapter;

   @Override
   public void play(String audioType, String fileName) {

      //inbuilt support to play mp3 music files
      if(audioType.equalsIgnoreCase("mp3")){
         System.out.println("Playing mp3 file. Name: " + fileName);
      }

      //mediaAdapter is providing support to play other file formats
      else if(audioType.equalsIgnoreCase("vlc") || audioType.equalsIgnoreCase("mp4")){
         mediaAdapter = new MediaAdapter(audioType);
         mediaAdapter.play(audioType, fileName);
      }

      else{
         System.out.println("Invalid media. " + audioType + " format not supported");
      }
   }
}
```

INTENT

Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.

APPLICABILITY

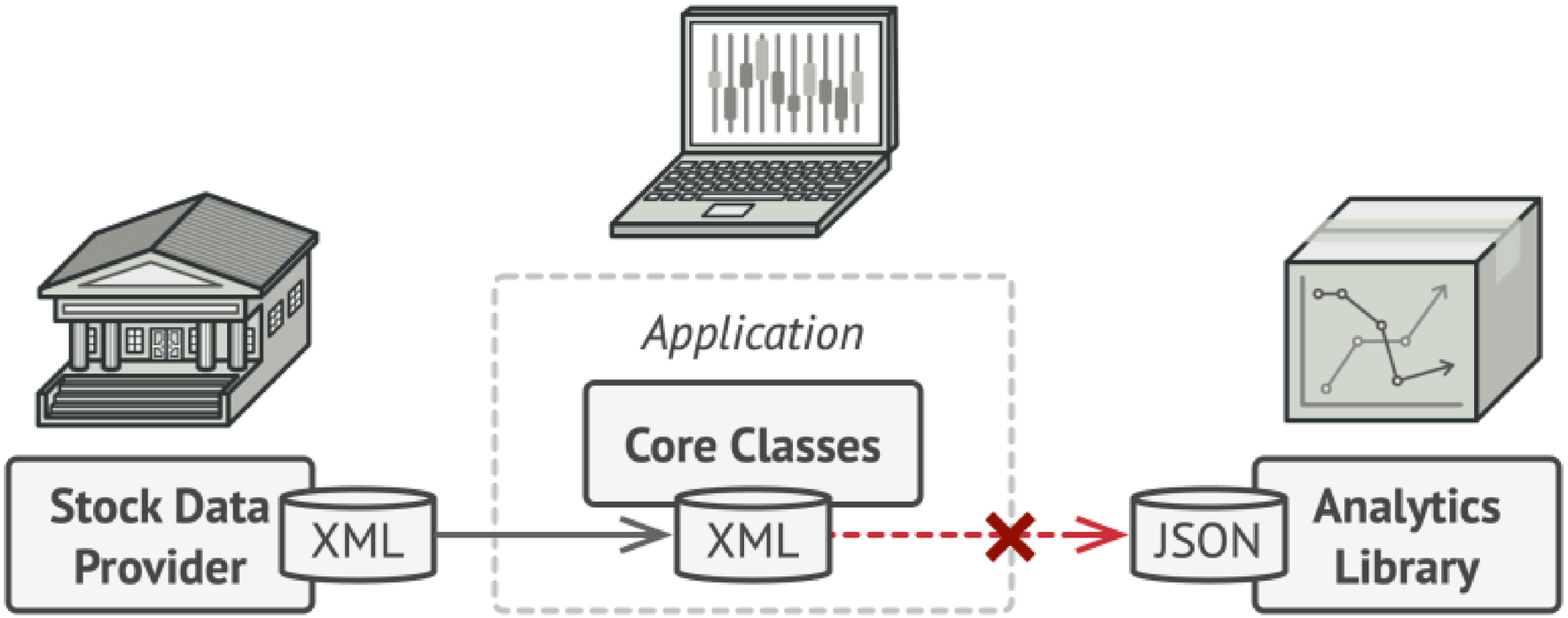
1. Use the Adapter class when you want to use some existing class, but its interface isn’t compatible with the rest of your code.
2. Use the pattern when you want to reuse several existing subclasses that. lack some common functionality that can’t be added to the superclass.

PROBLEM EXAMPLE

Imagine that you're creating a stock market monitoring app. The app downloads the stock data from multiple sources in XML format and then displays nice-looking charts and diagrams for the user.

At some point, you decide to improve the app by integrating a smart 3rd-party analytics library. But there's a catch: the analytics library only works with data in JSON format.

PROBLEM EXAMPLE



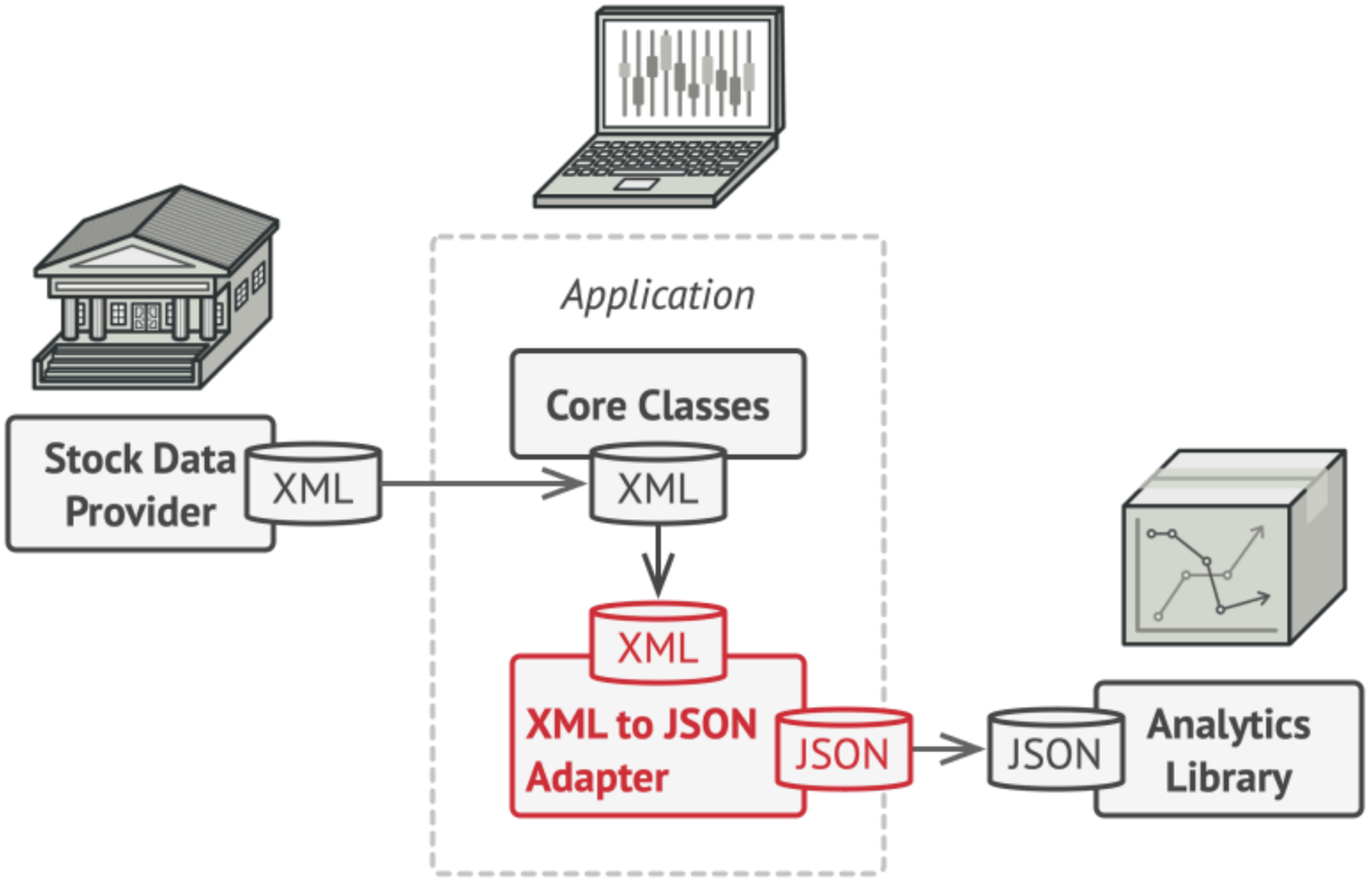
SOLUTION

You can create an adapter. This is a special object that converts the interface of one object so that another object can understand it.

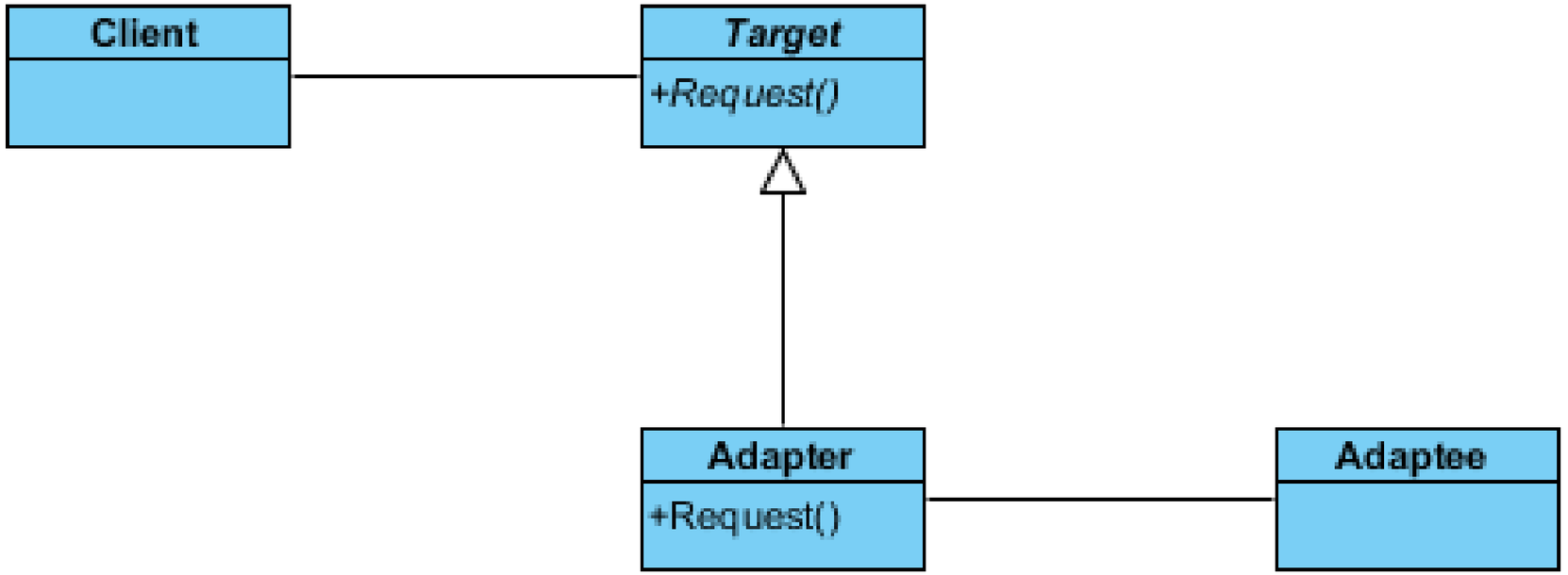
Adapters can not only convert data into various formats but can also help objects with different interfaces collaborate. Here’s how it works:

1. The adapter gets an interface, compatible with one of the existing objects.
2. Using this interface, the existing object can safely call the adapter’s methods.
3. Upon receiving a call, the adapter passes the request to the second object, but in a format and order that the second object expects.

SOLUTION


Application
Core Classes
XML
Stock Data Provider
XML
XML
XML to JSON Adapter
JSON
JSON
Analytics Library

ADAPTER DESIGN PATTERN UML DIAGRAM



THANK YOU